

$$\mathbf{z}_{1:T}^* = [\mathbf{z}_{1:(T/2)}^* \quad \mathbf{z}_{(T/2):T}^*]:$$

$$\frac{\partial \ell}{\partial (\cdot)} = \frac{\partial \ell}{\partial \mathbf{z}_{(T/2):T}^*} \underbrace{\frac{\partial \mathbf{z}_{(T/2):T}^*}{\partial (\cdot)}}_{(A)} + \frac{\partial \ell}{\partial \mathbf{z}_{(T/2):T}^*} \underbrace{\frac{\partial \mathbf{z}_{(T/2):T}^*}{\partial \mathbf{z}_{1:(T/2)}^*}}_{(B)} \underbrace{\frac{d\mathbf{z}_{1:(T/2)}^*}{d(\cdot)}}_{(C)} \quad (22)$$

where terms (A) and (B) require one evaluation of $f_\theta(\mathbf{z}_{(T/2):T}^*; [\mathbf{x}_{(T/2):T}, \mathbf{z}_{1:(T/2)}^*])$ and term (C) requires one evaluation of $f_\theta(\mathbf{z}_{1:(T/2)}^*; \mathbf{x}_{1:(T/2)})$. Hence, the memory cost is equivalent to that of applying f_θ once on the entire $\mathbf{z}_{1:T}^*$ (but with the subsequences' equilibrium likely easier to optimize).

F Task Descriptions

We briefly introduce the three sequence prediction tasks/datasets that we employ to evaluate the DEQ approach in Section 5.

Copy memory task. The copy memory task is a small but challenging synthetic stress test that has been frequently used in prior work to test a sequence model's memory retention ability [53, 4, 7]. In this task, each sequence $\mathbf{x}_{1:(T+20)}$ is 1-dimensional and has length $T + 20$, with $\mathbf{x}_{1:10}$ randomly selected from integers $1, 2, \dots, 8$ (with repetition). The rest of the input elements are all filled with zeros, except for the $\mathbf{x}_{T+10} = 9$. The goal of this task is to produce $\mathbf{y}_{1:(T+20)}$ such that $\mathbf{y}_{1:T+10} = \mathbf{0}$ and $\mathbf{y}_{T+11:T+20} = \mathbf{x}_{1:10}$. In other words, a sequence model trained on this task is expected to "recall" the first 10 elements of the sequence once it sees the delimiter $\mathbf{x}_{T+10} = 9$, and copy them to the end of the sequence. We generate 20K training samples and 2K testing samples. In prior works, [7] have shown that RNNs generally struggle with the task, especially when $T > 100$, whereas feedforward models tend to have better memory.

Penn Treebank. The Penn Treebank (PTB) corpus [31] is a commonly used dataset for character- and word-level language modeling. When used for word-level language modeling, PTB contains about 888K words at training, with a vocabulary size of 10,000. As this is a comparatively small language corpus (with punctuations and capitalization removed), prior work has shown that well-designed regularizations are required for best results [34, 52].

WikiText-103. The training corpus of WikiText-103 (WT103) [35] is about 110 times larger than PTB, with a vocabulary size over 260K. In general, this dataset is considered much more realistic than many others because it contains many rare words and retains punctuation, numbers, and capitalization from the original Wikipedia articles. WT103 is thus used to evaluate how well a sequence model scales to long sequences from a large vocabulary. This dataset has been frequently used in recent work with high-capacity sequence models [9, 8, 16, 6].